Week2

Real-Time Applications of Particle Swarm Optimization (PSO)

Job Scheduling – Allocating jobs to machines to minimize execution time.

1. Initialize parameters:
   - N = Swarm size (number of particles)
   - T_max = Maximum number of iterations
   - w = Inertia weight
   - c1, c2 = Cognitive and social acceleration coefficients
   - Jobs J = {j1, j2, ..., jn}, Machines M = {m1, m2, ..., mk}

2. Initialize Swarm:
   For each particle i in 1 to N:
      Initialize particle position X_i (job-to-machine mapping or priority sequence)
      Initialize particle velocity V_i
      Set personal best position pBest_i = X_i
      Evaluate fitness(X_i) -> makespan (total execution time)

3. Identify Global Best:
   gBest = particle position with the minimum fitness (minimum makespan) in the swarm

4. Main Loop (For iteration t = 1 to T_max):
   For each particle i in 1 to N:
      a. Update velocity:
         V_i = w * V_i + c1 * r1 * (pBest_i - X_i) + c2 * r2 * (gBest - X_i)
         (Note: For discrete job scheduling, map/round continuous velocity to valid machine/job permutations)

      b. Update position:
         X_i = X_i + V_i

      c. Decode and evaluate fitness:
         Map X_i to a valid schedule on machines M
         Calculate fitness(X_i) = Makespan (completion time of the latest job)

      d. Update personal best (pBest_i):
         If fitness(X_i) < fitness(pBest_i):
            pBest_i = X_i

   e. Update global best (gBest):

gBest = particle with the minimum fitness among all pBest_i

5. End Loop

6. Return gBest as the optimal or near-optimal job schedule and its minimum makespan.

```
import numpy as np

# ------------------------------------------------------------
# SETUP & PROBLEM CONFIGURATION
# ------------------------------------------------------------
# Execution time matrix: Row = Machine, Column = Job
# Processing time of Job J on Machine M
processing_times = np.array([
    [4, 8, 3, 7, 5],  # Machine 0 processing times for jobs 0-4
    [6, 3, 9, 2, 8],  # Machine 1 processing times for jobs 0-4
    [5, 7, 4, 6, 3]   # Machine 2 processing times for jobs 0-4
])

num_machines, num_jobs = processing_times.shape

# ------------------------------------------------------------
# FITNESS FUNCTION (MAKESPAN CALCULATOR)
# ------------------------------------------------------------
def calculate_makespan(position):
    """
    Decodes the particle position into a job-to-machine mapping
    and calculates the total execution time (makespan).
    """
    # Map continuous float values to discrete machine IDs (0, 1, or 2)
    # Using modulo ensures values stay within valid machine indices
    machine_assignments = np.floor(position).astype(int) % num_machines

    # Track the accumulated load/time on each machine
    machine_loads = np.zeros(num_machines)

    for job_idx, machine_idx in enumerate(machine_assignments):
```

```
        machine_loads[machine_idx] += processing_times[machine_idx,
job_idx]

    # The objective is to minimize the maximum time any single machine
takes
    return np.max(machine_loads), machine_assignments

# ------------------------------------------------------------
# PSO PARAMETERS
# ------------------------------------------------------------
num_particles = 15
max_iterations = 50
w = 0.7   # Inertia weight
c1 = 1.5  # Cognitive coefficient (personal memory)
c2 = 1.5  # Social coefficient (swarm memory)

# Initialize positions and velocities
# Positions are initialized in a range matching our machine count
X = np.random.uniform(0, num_machines, (num_particles, num_jobs))
V = np.random.uniform(-1, 1, (num_particles, num_jobs))

# Evaluate initial states
pBest_X = np.copy(X)
pBest_fitness = np.zeros(num_particles)
for i in range(num_particles):
    pBest_fitness[i], _ = calculate_makespan(X[i])

# Identify global best
gBest_idx = np.argmin(pBest_fitness)
gBest_X = np.copy(pBest_X[gBest_idx])
gBest_fitness = pBest_fitness[gBest_idx]

# ------------------------------------------------------------
# MAIN PSO OPTIMIZATION LOOP
# ------------------------------------------------------------
for t in range(max_iterations):
    for i in range(num_particles):
        r1, r2 = np.random.rand(), np.random.rand()

        # 1. Update Velocity
```

```
            V[i] = (w * V[i] +
                    c1 * r1 * (pBest_X[i] - X[i]) +
                    c2 * r2 * (gBest_X - X[i]))

            # 2. Update Position
            X[i] = X[i] + V[i]

            # 3. Evaluate Fitness
            current_fitness, _ = calculate_makespan(X[i])

            # 4. Update Personal Best (pBest)
            if current_fitness < pBest_fitness[i]:
                pBest_fitness[i] = current_fitness
                pBest_X[i] = np.copy(X[i])

        # 5. Update Global Best (gBest)
        min_current_idx = np.argmin(pBest_fitness)
        if pBest_fitness[min_current_idx] < gBest_fitness:
            gBest_fitness = pBest_fitness[min_current_idx]
            gBest_X = np.copy(pBest_X[min_current_idx])

# Final decoding of the optimal arrangement
optimal_makespan, optimal_schedule = calculate_makespan(gBest_X)

print(f"Optimal Makespan (Min Time): {optimal_makespan} units")
for job, machine in enumerate(optimal_schedule):
    print(f" -> Job {job} is assigned to Machine {machine}")
```

```
Optimal Makespan (Min Time): 8.0 units
 -> Job 0 is assigned to Machine 1
 -> Job 1 is assigned to Machine 2
 -> Job 2 is assigned to Machine 0
 -> Job 3 is assigned to Machine 1
 -> Job 4 is assigned to Machine 0
```